

ILCMS Platform Version 1 — Consolidated Developer Handoff

Requirements & Design Baseline derived from Technical Specifications v0.1–v1.9

2 September 2026

Developer handoff note: This is a consolidated implementation baseline, not a verbatim concatenation of individual specification files. It resolves the v0.1–v1.9 material into a single Version 1 build target.

Version Map

Version	Primary concern
v0.1–v0.8	Product requirements, domain, Icelandic context and architecture
v0.9	Frontend architecture & UX
v1.0	Consolidated platform architecture
v1.1	DevOps, CI/CD & Kubernetes
v1.2	Backend implementation
v1.3	Database & data model
v1.4	REST API & OpenAPI
v1.5	Authentication, authorization & security
v1.6	Documents, electronic records & archiving
v1.7	Case management, workflow, tasks, deadlines & legal process
v1.8	Search, notifications, calendars, reporting & collaboration
v1.9	Administration, configuration, master data & system management

1. Executive Summary

- ILCMS (Icelandic Legal Case Management System) is a secure, configurable, Iceland-focused legal case-management platform. Version 1 is the implementation baseline produced by consolidating the requirements and architecture developed through specifications v0.1–v1.9.
- The platform is designed around the Case as the central operational object. Cases connect parties, documents, tasks, deadlines, hearings, proceedings, correspondence, notes, audit events and reporting. Search, notifications, calendars, reporting and administration are supporting services around this authoritative domain model.
- The system is not intended to make autonomous legal decisions. It may organize information, calculate configured deadlines, identify potential conflicts, extract information and assist users, but legally consequential decisions remain under human control.

2. Version 1 Scope

- Version 1 shall provide a production-capable foundation for an Icelandic legal organization, including identity and authorization, organization management, case management, parties, documents and records, workflows, tasks, deadlines, search, notifications, calendars, reporting, collaboration, administration, auditability and secure deployment.
- AI/LLM functionality is deliberately kept behind an integration boundary. Advanced AI/RAG/legal-intelligence capabilities are candidates for Version 2 and must not be required for core Version 1 operation.

3. Specification Traceability

- v0.1–v0.8 established the product concept, requirements, Icelandic legal/records-management context, domain model and overall architecture. v0.9 defined frontend architecture and UX. v1.0 established the consolidated platform architecture and implementation direction. v1.1 defined DevOps, CI/CD and Kubernetes. v1.2 defined backend implementation. v1.3 defined database/data-model requirements. v1.4 defined REST/OpenAPI. v1.5 defined authentication, authorization and security. v1.6

defined documents, electronic records and archiving. v1.7 defined case management, workflow, tasks, deadlines and legal process. v1.8 defined search, notifications, calendars, reporting and collaboration. v1.9 defined administration, configuration, master data and system management.

4. Core Architectural Principles

- Security by default; explicit server-side authorization; strong organization/tenant isolation.
- Case and document data are authoritative. Search indexes, notifications, caches and derived analytics are not authoritative.
- Domain logic belongs in services/domain modules, not API controllers.
- Database integrity is a first-class control.
- Sensitive operations are auditable and audit records are append-only.
- Long-running work is asynchronous and idempotent.
- External providers are accessed through adapters.
- Configuration is versioned, validated and auditable.
- Icelandic UTF-8 names, legal terminology and date/time handling are first-class requirements.
- Core operation must support private-cloud, on-premises and fully air-gapped deployment.

5. Target Technology Stack

- Backend: Python 3.12+, FastAPI, Uvicorn, SQLAlchemy 2.x, Pydantic v2, Alembic.
- Database: PostgreSQL. Cache: Redis for transient/cache purposes only. Messaging: RabbitMQ. Object storage: S3-compatible storage. Search: PostgreSQL full-text search for MVP with an abstraction for OpenSearch/Elasticsearch-compatible search later.
- Frontend: modern web application consuming the versioned REST API. The frontend shall implement responsive case-centric UX, accessible components, global search, dashboards, document views, task/deadline management, calendar and administration.
- Infrastructure: Docker containers, Kubernetes, Helm/GitOps, private container registry, Vault/secret management, TLS ingress, observability with Prometheus/Grafana/OpenTelemetry-compatible tooling, centralized structured logs, automated backup and restore procedures.

6. Logical Architecture

- The preferred deployment is a modular service-oriented application rather than a prematurely fragmented microservice system. A single backend application may initially contain clearly separated domain modules while workers run independently.
- Logical flow: Frontend → API/Authentication → Domain & Application Services → Repositories → PostgreSQL/Object Storage. Domain mutations create audit events and outbox events transactionally. RabbitMQ distributes asynchronous work to document, notification, search, reporting, calendar and future AI workers.
- Redis is never the authoritative store. Search indexes and derived reporting data must be rebuildable from authoritative data.

7. Repository Structure

- Recommended repository layout:
- backend/ app/ api/ domain/ services/ repositories/ models/ schemas/ auth/ workers/ migrations/ tests/
- frontend/ src/ components/ pages/ services/ state/ tests/
- deploy/ helm/ k8s/ terraform/
- docs/ architecture/ api/ operations/ security/
- scripts/
- API controllers should validate input, establish authorization context and call application services. They should not contain business rules or direct database queries.

8. Identity, Organizations & Authorization

- Authentication should preferably use an OIDC-compatible identity provider. The backend validates issuer, audience, signature, expiry, required claims, organization membership and authentication assurance where required.
- RBAC is the initial authorization model, with granular permissions. Example roles include SYSTEM_ADMIN, ORGANIZATION_ADMIN, LAWYER, PARALEGAL, CASE_MANAGER, ASSISTANT, READ_ONLY and AUDITOR.
- RBAC alone is insufficient. Every protected resource requires object-level authorization using user + organization + role + permission + resource + resource ownership/access rules.
- Administrative privileges must not automatically grant access to confidential legal case content. System administration and legal-data access are separate concerns.
- PostgreSQL row-level security may be used as a defense-in-depth tenant isolation mechanism. Authorization must remain enforced at the application layer.

9. Core Data Model

- Case is the central entity. Suggested fields: id, organization_id, case_number, title, description, case_type, classification_id, status, priority, confidentiality_level, opened_at, closed_at, owner_user_id, workflow_id, workflow_version_id, created_by, created_at, updated_at.
- Cases have multiple parties through a case_parties relationship. Party types include PERSON, ORGANIZATION, AUTHORITY, COURT and OTHER. Icelandic names must not assume a mandatory Western surname model; support display_name, given_name, family_name and full_legal_name.
- Cases may contain assignments, documents, tasks, deadlines, hearings, proceedings, court events, communications, notes, timeline events, time entries, expenses and relationships.
- Case relationships include RELATED_TO, PARENT_OF, CHILD_OF, DUPLICATE_OF, MERGED_FROM, SPLIT_FROM and APPEAL_OF.

10. Case Lifecycle & Legal Process

- Primary case states: INTAKE, OPEN, ACTIVE, PENDING, RESOLVED, CLOSED, ARCHIVED. Optional states include REJECTED, CONFLICT_CHECK, ON_HOLD, SUSPENDED and REOPENED.
- Case workflow state is separate from overall case status. A case can be ACTIVE while its workflow state is AWAITING_DEFENDANT_RESPONSE.
- Proceedings are separate from cases so one case can contain district-court, appeal and Supreme Court proceedings. Each proceeding may have its own court and court case number.
- Closure uses a controlled process such as ACTIVE → RESOLVED → CLOSURE_REVIEW → CLOSED. Closure checks may include mandatory tasks, critical deadlines, required metadata/documents, billing, client communication, retention policy and legal holds.

11. Conflict Checking

- Conflict checking is an identification and review process. Search should cover clients, opposing parties, related organizations and aliases. Results: NO_CONFLICT_FOUND, POTENTIAL_CONFLICT, CONFLICT_CONFIRMED and REVIEW_REQUIRED.
- The system identifies possible conflicts; an authorized legal professional makes the final legal determination. Conflict searches and decisions are audited.

12. Documents, Records & Archiving

- Documents are first-class records linked to cases and metadata. Support versioning, secure object storage, metadata, access control, virus scanning, OCR/text extraction, indexing and retention.
- Document processing pipeline: upload → validation → virus scan → metadata → OCR/text extraction → classification/extraction where configured → search indexing → READY. Processing failures are explicit and retryable.
- The system must distinguish original document, extracted text, OCR output, AI-generated summary and human-approved summary. Derived text is not a replacement for the original record.
- Support records-management concepts including classification/málalykill, case registration metadata, retention policies, legal holds, controlled disposition and archival export. Exact legal obligations must remain configurable and be validated against current Icelandic requirements before production use.

- Normal case/document deletion should generally be avoided. Physical destruction is controlled, retention-aware, legal-hold-aware, authorized and audited.

13. Tasks, Deadlines & Calendars

- Tasks: TODO, IN_PROGRESS, BLOCKED, COMPLETED, CANCELLED. Support assignment, priorities, dependencies, templates, recurring generation and completion history.
- Deadlines are distinct from ordinary tasks. Types include COURT, STATUTORY, CONTRACTUAL, INTERNAL, CLIENT, ADMINISTRATIVE and OTHER.
- Deadline calculation rules must store their inputs and be independently testable. Rules may depend on calendar days, working days, weekends, holidays, court rules, triggering events and receipt dates.
- Use timezone-aware timestamps. PostgreSQL TIMESTAMPTZ is preferred. Canonical Iceland timezone: Atlantic/Reykjavik. Store UTC where appropriate while preserving intended event timezone.
- Manual deadline overrides require new value, reason, actor and timestamp while retaining the original calculated value. AI-extracted deadlines require human review before confirmation.

14. Workflow & Automation

- Workflows consist of versioned states, transitions, task templates, deadline templates, notifications and required documents. Existing active cases normally remain on their original workflow version unless explicitly migrated.
- Transitions are explicit and authorization-controlled. Configurable transition validation may require completion of mandatory tasks, resolution of deadlines, required documents or absence of legal holds.
- Automation rules must be deterministic, auditable, idempotent, versioned and disableable. Automation may coordinate work but must not make legal decisions.

15. REST API & OpenAPI

- Public API base path: /api/v1. REST/JSON over HTTPS. OpenAPI 3.x documentation generated by FastAPI.
- Use plural resource nouns, UUID technical IDs, conventional HTTP semantics, pagination, whitelisted filtering/sorting, RFC3339 timestamps and stable machine-readable enums.
- Standard error shape: { error: { code, message, request_id } }. Never expose stack traces, database errors or infrastructure details.
- Use X-Request-ID for correlation. Use Idempotency-Key for retryable mutations and long-running operations. Use ETag/If-Match or equivalent optimistic concurrency where appropriate.
- Large/expensive operations return 202 Accepted and create background jobs.

16. Search

- Global search covers cases, documents, document content, parties, proceedings, court case numbers, tasks, deadlines, hearings, communications and permitted notes.
- MVP search may use PostgreSQL tsvector/GIN/pg_trgm. The application must isolate search behind a provider abstraction for later OpenSearch/Elasticsearch-compatible deployment.
- Search must be authorization-aware and must never become an access-control bypass. Icelandic characters, names, Unicode normalization, case-insensitivity and appropriate linguistic processing must be tested.
- Search indexes are derived and rebuildable. Reindexing is asynchronous, resumable and audited.

17. Notifications

- Notifications are derived from authoritative events. Channels: in-app, email, optional push and future enterprise integrations.
- Support task assignments, deadline reminders/overdue events, case assignments/status changes, document processing, hearings, approvals, conflict review, court events and legal-hold alerts.
- Notification delivery uses RabbitMQ workers, retries, exponential backoff/dead-letter handling and idempotency keys. Preferences are configurable subject to mandatory organizational alerts.
- External notifications must minimize confidential content, particularly in email subjects/previews and lock-screen push notifications.

18. Calendar & External Synchronization

- Calendar views: day, week, month and agenda. Personal, team and case calendars are supported.
- Authoritative events include hearings, legal deadlines and case events. External calendars are synchronization targets and must never become the authoritative source for legal deadlines.
- Support adapters for Microsoft 365/Outlook, Google Calendar and CalDAV where required. External credentials are stored in secret management, not normal configuration.
- Synchronization failures must be isolated from core case management. External calendar entries should contain minimal confidential information.

19. Reporting & Dashboards

- Reports include cases by status/type/classification/lawyer/team/date, upcoming and overdue deadlines, tasks, workload, proceedings and case aging.
- Reports are authorization-aware. Large exports are asynchronous and should support CSV, XLSX and PDF where appropriate.
- Reporting should use optimized views/materialized views or a separate reporting database at scale. Reports must not overload transactional case-management queries.
- Sensitive report generation/export should be audited with actor, filters, time and result count.

20. Collaboration

- Collaboration is case-centric: assignments, tasks, documents, notes/comments, approvals, communications and activity feeds.
- Comments and notes have explicit visibility such as CASE_TEAM, ORGANIZATION and PRIVATE. Private notes must not become visible merely because case access changes.
- Real-time updates via WebSockets/SSE are optional and non-authoritative. Optimistic concurrency protects against overwriting another user's changes.

21. Administration & Configuration

- Organizations, departments, teams, users, roles, permissions, case types, classifications, workflows, task templates, deadline rules, calendars, notification rules, document categories, retention policies, custom fields, numbering schemes, integrations and feature flags are configurable.
- Configuration is versioned, validated, audited and preferably approved for high-impact changes. Published versions should not be edited in place.
- Configuration promotion should support Development → Test → Staging → Production and declarative export/import using JSON/YAML. Secrets and case content must never be included in configuration exports.
- Configuration dependency checks must prevent deletion of referenced objects. Prefer deactivation over physical deletion.

22. Audit & Security

- Audit events are append-only and should include event ID, organization, actor, event type, resource type/id, timestamp, request ID and technical metadata. Sensitive operations include authentication events, case/document access changes, exports, administrative changes, retention/legal-hold operations and configuration changes.
- Audit storage should be tamper-evident and/or exported to immutable/WORM-compatible storage where required.
- Security baseline: no hard-coded credentials; dependency/container/SAST scanning; authentication/authorization tests; tenant isolation and IDOR/BOLA tests; file-upload security; sensitive-log review; TLS; encryption; backup/restore and disaster-recovery testing.
- Break-glass administration, if implemented, requires reason, elevated authentication, high-priority audit, optional security notification and post-event review.

23. Async Processing & Messaging

- RabbitMQ handles asynchronous jobs/events. Typical workers: document, notification, search-index, reporting, calendar and future AI.

- Message envelope should contain event_id, event_type, occurred_at, organization_id, resource_id and schema_version. Avoid confidential payloads; use identifiers/references.
- Consumer flow: receive → validate → authorization/context check → process → commit → acknowledge. Retries and dead-letter queues are required.
- Important mutations use transactional outbox: database state + audit event + outbox event commit together. Consumers must tolerate duplicate delivery.

24. Observability & Operations

- Use structured JSON logs, metrics and distributed tracing. Do not put document content, passwords, tokens or unnecessary legal-case content in normal logs.
- Monitor API latency/errors, database health, queue depth, worker failures, search-index lag, notification failures, calendar-sync failures, document-processing backlog, storage usage and report-generation time.
- Health endpoints must distinguish liveness and readiness. Administrative health dashboards must not expose confidential case data.

25. Deployment & Infrastructure

- Use Docker images with immutable tags/digests, private registry, Kubernetes Deployments/Services, HPA, PDB, NetworkPolicies, ingress/TLS and Helm/GitOps deployment.
- Use PostgreSQL with a production HA strategy where required; PgBouncer may be used for connection pooling. Use S3-compatible object storage with versioning, encryption, access controls and audit capabilities.
- Use Vault/External Secrets/CSI or an equivalent secret-management architecture. Terraform manages infrastructure. GitOps manages deployment state.
- Core platform must operate on-premises, private cloud and air-gapped environments. External identity, calendar, email, search and AI providers are adapters rather than architectural dependencies.

26. Frontend Requirements

- The UI should be case-centric and optimized for legal work: dashboard, global search, case list/detail, parties, documents, workflow, tasks, deadlines, hearings, proceedings, timeline, notifications, calendar, reports and administration.
- Accessibility, keyboard navigation, screen-reader support, responsive layouts and Icelandic UTF-8 rendering are required.
- Use stable backend IDs and machine enums while presenting localized Icelandic/English terminology. Avoid hard-coded business rules in the frontend.

27. Icelandic Requirements

- Use UTF-8 throughout HTTP, JSON, PostgreSQL, object metadata, messaging, search, logs and exports. Test names and organizations containing Á É Í Ó Ú Ý Þ Æ Ö Ð and lowercase equivalents.
- Do not assume Icelandic names have a conventional family name. Preserve legal names exactly as entered/received while providing appropriate search/sort behavior.
- Support Icelandic legal/records-management concepts including mál, málaskrá, málalykill and retention/archiving mechanisms. Court-document workflows should support descriptive/datable filenames, searchable PDFs/OCR where applicable and separate electronic files per document as configurable workflow requirements.
- Legal and records-management configuration must be validated against current Icelandic law, court rules and Þjóðskjalasafn requirements before production. This document is a technical specification, not legal advice.

28. Data Protection & Privacy

- Apply data minimization, least privilege, purpose limitation, retention controls and secure deletion/disposition. Confidentiality is a case-level attribute and can be further constrained by object/field permissions.
- Minimize confidential content in external systems, logs, metrics, notifications and calendar integrations. Do not transmit case content to external AI services unless explicitly configured and authorized.
- Backups and derived data must follow the organization's retention/security policies. Search indexes and caches must be rebuildable where possible.

29. Testing Strategy

- Unit tests: domain rules, deadline calculations, workflow transitions, authorization decisions, configuration validation.
- Integration tests: PostgreSQL/Alembic, object storage, RabbitMQ, search, authentication provider, notification delivery and calendar adapters.
- Security tests: cross-tenant access, BOLA/IDOR, privilege escalation, deactivated users, confidential cases, private notes, export access, file uploads and secret exposure.
- End-to-end journeys: login → create intake → conflict check → open case → upload/process document → assign task → calculate deadline → notify → schedule hearing → search → report → close/archive.

30. Version 1 Implementation Plan

- Phase 0 — Engineering foundation: repository, CI, containers, configuration, FastAPI skeleton, PostgreSQL/Alembic, health checks, logging, secrets and Kubernetes deployment.
- Phase 1 — Identity & organization: OIDC, users, organizations, teams, roles, permissions, tenant isolation and audit.
- Phase 2 — Core case management: cases, parties, classifications, assignments, status lifecycle, conflict checking and case timeline.
- Phase 3 — Documents & records: object storage, upload security, metadata, versions, OCR/text extraction, retention/legal holds and document processing.
- Phase 4 — Work management: workflows, tasks, dependencies, deadline rules, calendars, reminders and background jobs.
- Phase 5 — Search & collaboration: global search, indexing, notifications, activity feed, comments/subscriptions and dashboards.
- Phase 6 — Reporting & administration: reports, exports, configuration UI, master data, workflow designer, policies and system administration.
- Phase 7 — Production hardening: load testing, penetration testing, backup/restore, DR exercises, operational runbooks, security review and Icelandic legal/records-management validation.

31. Initial MVP Acceptance Checklist

- Authentication and organization isolation work.
- RBAC plus object-level authorization is enforced on every protected resource.
- Cases can be created, classified, assigned, searched, updated, resolved and closed.
- Parties and Icelandic names are supported correctly.
- Documents can be securely uploaded, versioned, processed and retrieved.
- Tasks and legally significant deadlines are distinct and auditable.
- Workflow transitions and deadline calculations are configurable and tested.
- Search returns only authorized results.
- Notifications are asynchronous, retryable and deduplicated.
- Calendar views work and external synchronization is isolated.
- Reports respect authorization and large exports are asynchronous.
- Administrative configuration is versioned, audited and protected.
- Audit events are append-only and security events are captured.
- Backups/restores and disaster recovery have been tested.
- Core functionality works without external cloud/AI dependencies in an air-gapped deployment.

32. Developer Definition of Done

- A feature is not complete merely because its API works. It is complete when its domain logic, database migration, authorization, audit behavior, API schema, frontend integration, tests, observability, documentation and deployment configuration are implemented.

- Every production feature should answer: Who can use it? What data can they see? What is authoritative? What is audited? What happens on retry/failure? What happens on deletion/retention? How is it localized? How does it operate offline/air-gapped? How is it tested?
- No feature should introduce a provider dependency directly into domain services, expose sensitive data through logs/errors, bypass the authorization service, or silently modify historical legal records.

33. Key API Resource Inventory

- Core: /api/v1/me, /organizations, /users, /cases, /parties, /documents, /tasks, /deadlines, /hearings, /proceedings, /communications, /notes, /notifications, /calendar, /reports, /search, /audit, /jobs.
- Administration: /api/v1/admin/organizations, /users, /teams, /roles, /permissions, /case-types, /classifications, /workflows, /templates, /deadline-rules, /notification-rules, /document-categories, /retention-policies, /integrations, /configuration, /jobs, /health.
- All public endpoints are versioned and documented in OpenAPI. Exact endpoint names may be refined during API implementation provided the principles and backward-compatibility requirements remain intact.

34. Final Platform Principle

- ILCMS Version 1 is a secure, configurable, auditable system of record and work coordination for legal cases. PostgreSQL/object storage hold authoritative business and document data; the API is the controlled application boundary; workers provide asynchronous processing; search, notifications, calendars and reporting are derived/supporting services; administration controls behavior through versioned configuration; and all access is governed by explicit authorization.
- The architecture intentionally separates legal records from derived intelligence and separates technical administration from access to legal content. This provides a foundation on which Version 2 can safely add private/local LLMs, RAG, AI-assisted legal workflows, advanced integrations and analytics without compromising the core case-management model.

Appendix A — Developer Priority Matrix

Priority	Build first	Reason
P0	Identity, tenant isolation, DB, audit, API foundation	Security and data integrity prerequisites
P0	Cases, parties, authorization, documents	Core legal workflow
P0	Tasks, deadlines, workflow, background jobs	Operational case management
P1	Search, notifications, calendar	Daily usability and work coordination
P1	Administration/configuration	Makes behavior maintainable and organization-specific
P1	Reporting/dashboard	Operational management
P2	External calendar integrations	Useful but not required for core operation
P2	Advanced analytics	Can follow transactional foundation
V2	AI/RAG/legal intelligence	Requires separate governance and AI architecture

Appendix B — Non-Negotiable Engineering Rules

- Never bypass server-side authorization.
- Never use search as an authorization mechanism.
- Never store secrets in source code or ordinary configuration.
- Never make a derived index/cache the only copy of authoritative data.
- Never silently rewrite historical legal records.
- Never put sensitive case/document content in ordinary logs or external notifications unnecessarily.
- Never make external cloud services mandatory for core case operation.
- Never allow an AI result to become authoritative legal data without explicit human action.
- All retryable asynchronous consumers must be idempotent.
- All high-impact mutations must be auditable.